

Exercise 5 – Hugging Face and Google Colab

1. Hugging Face Space — Text-to-Image Demo

Space used: SenseNova-U1.5-8B-MoT (unified text-to-image and image editing)

Prompt used: "A futuristic city skyline at night with flying vehicles, neon lights reflecting on wet streets, cyberpunk style"

Settings: Aspect ratio 16:9, Seed 1432621567

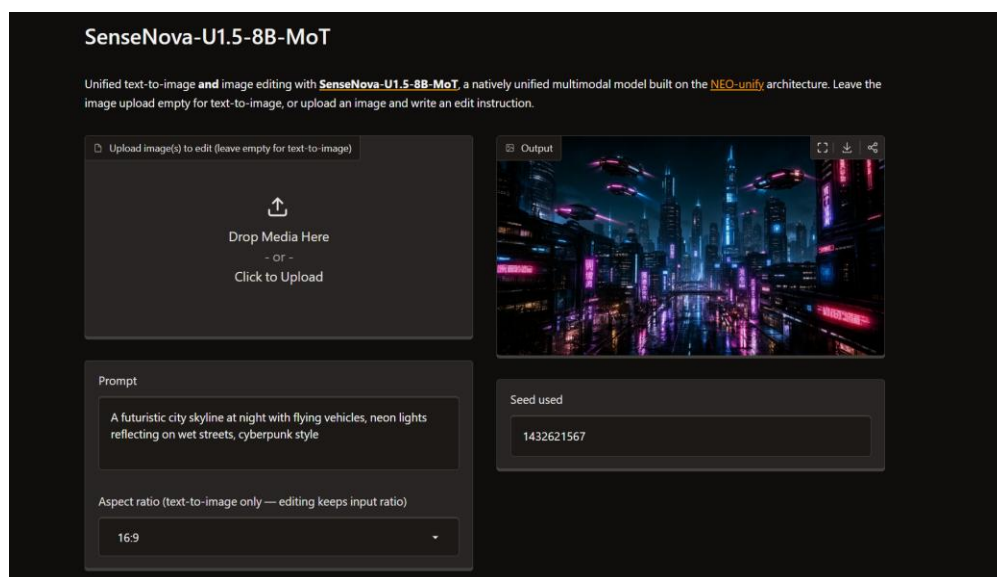


Figure 1: Generated cyberpunk city skyline using the SenseNova-U1.5-8B-MoT Space.

2. Exploring a Model on Hugging Face

Model explored: HauhouCS/Qwen3.8-27B-Uncensored-HauhouCS-Aggressive-MTP-GGUF (Image-Text-to-Text, GGUF quantized, Apache-2.0 license).

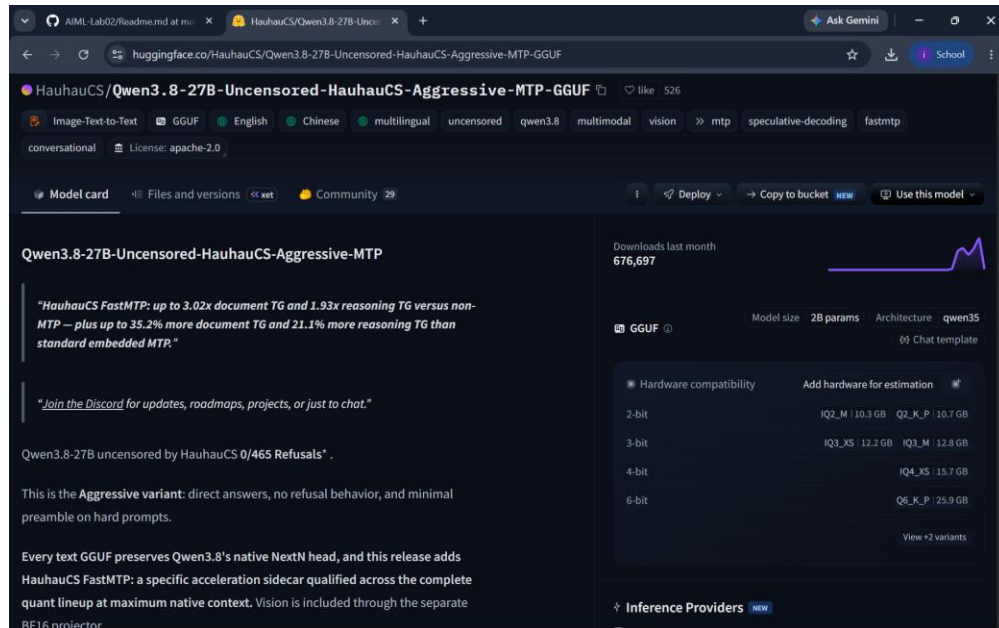


Figure 2: Model card overview, tags, and hardware compatibility table.

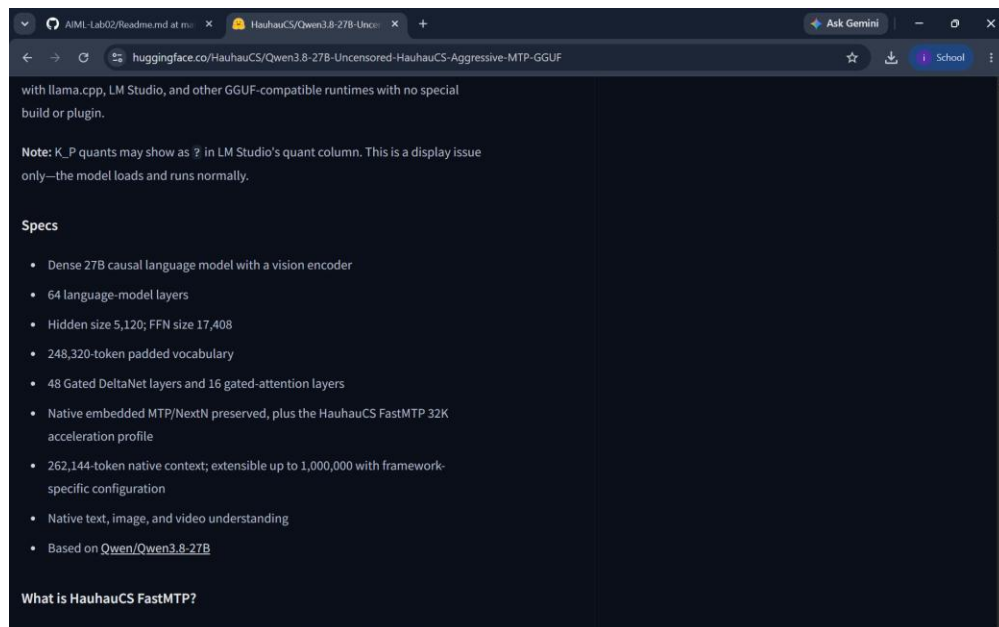


Figure 3: Model specs — 27B dense causal language model with vision encoder, 262K native context.

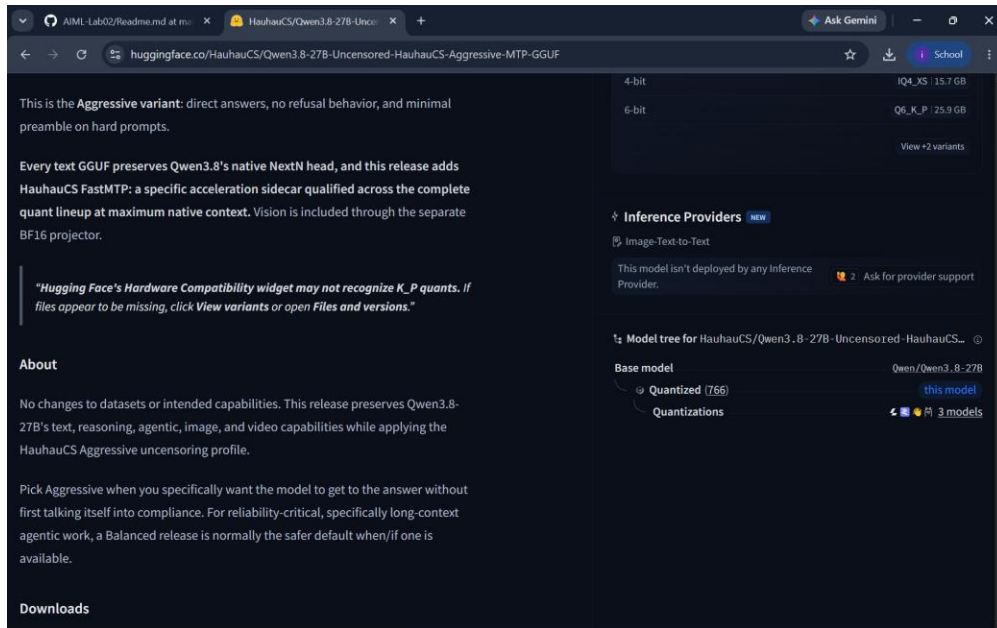


Figure 4: Model tree showing base model (Qwen3.8-27B) and quantization details.

4. Step 9 — General Summarization Pipeline

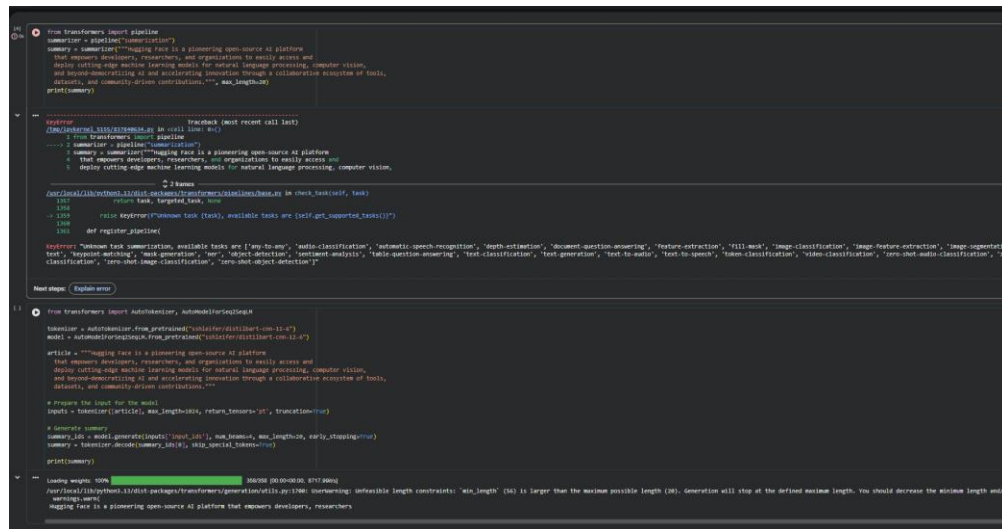


Figure 5: pipeline() KeyError (top) and the successful workaround using AutoTokenizer/AutoModelForSeq2SeqLM (bottom).

5. Step 11 — kobart-summary-v3 Model

```
from transformers import pipeline

# Use a known summarization model
summarizer = pipeline("summarization", model="EthanLee/kobart-summary-v3")

# Input text to summarize
text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access and
deploy cutting-edge machine learning models for natural language processing, computer vision,
and beyond—democratizing AI and accelerating innovation through a collaborative ecosystem of tools,
datasets, and community-driven contributions."""

# Generate summary
summary = summarizer(text, max_length=30, min_length=5, do_sample=False)
print(summary)

*** Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 168k/168k [00:00<00.00, 112kB/s]

KeyError                                Traceback (most recent call last)
/tmp/ipykernel_5155/3369716615.py in <cell line: 0>()
      2
----> 3 summarizer = pipeline("summarization", model="EthanLee/kobart-summary-v3")
      4
      5
      6 # Input text to summarize

2 frames
/usr/local/lib/python3.13/dist-packages/transformers/pipelines/base.py in check_task(self, task)
    1357         return task, targeted_task, None
    1358
-> 1359         raise KeyError(f"Unknown task {task}, available tasks are {self.get_supported_tasks()}")
    1360
    1361     def register_pipeline(

KeyError: "Unknown task summarization, available tasks are ['any-to-any', 'audio-classification', 'automatic-speech-recognition', 'depth-estimation', 'document-question-answering', 'feature-extraction', 'fill-mask', 'image-classification', 'image-feature-extraction', 'image-segmentation', 'image-text-to-text', 'keypoint-matching', 'mask-generation', 'ner', 'object-detection', 'sentiment-analysis', 'table-question-answering', 'text-classification', 'text-generation', 'text-to-audio', 'text-to-speech', 'token-classification', 'video-classification', 'zero-shot-audio-classification', 'zero-shot-classification', 'zero-shot-image-classification', 'zero-shot-object-detection']"
```

Figure 6: Same pipeline KeyError encountered when loading the kobart-summary-v3 model.

6. Step 12–13 — Model of Choice: HauhauCS Qwen3.8-27B (GGUF)

```
[2] !pip install -U llama-cpp-python

Collecting llama-cpp-python
  Using cached llama_cpp_python-0.3.35.tar.gz (74.9 MB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (4.16.0)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (2.1.3)
Collecting diskcache>=5.6.1 (from llama-cpp-python)
  Using cached diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: Jinja2>=2.11.3 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from Jinja2>=2.11.3->llama-cpp-python) (3.0)
Using cached diskcache-5.6.3-py3-none-any.whl (45 kB)
Building wheels for collected packages: llama-cpp-python
  Building wheel for llama-cpp-python (pyproject.toml) ... done
  Created wheel for llama-cpp-python: filename=llama_cpp_python-0.3.35-py3-none-linux_x86_64.whl size=21110761 sha256=6a63209255babc0fe
  Stored in directory: /root/.cache/pip/wheels/8e/41/ad/9e48f16395e89552458310ec94d372420d542fc811ce5b4465
Successfully built llama-cpp-python
Installing collected packages: diskcache, llama-cpp-python
Successfully installed diskcache-5.6.3 llama-cpp-python-0.3.35

Local Inference on GPU

Model page: https://huggingface.co/HauhauCS/Qwen3.8-27B-Uncensored-HauhauCS-Aggressive-MTP-GGUF

⚠ If the generated code snippets do not work, please open an issue on either the model repo and/or on huggingface.js

from llama_cpp import Llama

llm = Llama.from_pretrained(
    repo_id="HauhauCS/Qwen3.8-27B-Uncensored-HauhauCS-Aggressive-MTP-GGUF",
    filename="Qwen3.8-27B-Uncensored-HauhauCS-Aggressive-Q4_K_P.gguf",
)

/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_validators.py:205: UserWarning: The 'local_dir_use_symlinks' argument is deprecated.
warnings.warn(

/Qwen3.8-27B-Uncensored-HauhauCS-Aggres(...): reconstructing file: 100% 17.9GB / 17.9GB, 167MB/s

/Qwen3.8-27B-Uncensored-HauhauCS-Aggres(...): downloading bytes: 17.9GB, 119MB/s

llama_model_loader: loaded meta data with 54 key-value pairs and 866 tensors from /root/.cache/huggingface/hub/models--HauhauCS--Qwen3.8-27B-Uncensored-HauhauCS-Aggressive-Q4_K_P.gguf
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:                               general.architecture str           = qwen35
llama_model_loader: - kv 1:                               qwen35.full_attention_interval u32      = 4
llama_model_loader: - kv 2:                               qwen35.attention.key_length u32         = 256
```

Figure 7: Auto-generated Colab code for local GPU inference with the chosen model.

7. Reflection

The main challenge in this exercise was an environment-level issue where `pipeline()` could not resolve the "summarization" task despite it being a standard Hugging Face pipeline type — this was worked around by loading the tokenizer and model classes directly. Exploring a large, quantized, vision-capable model (27B parameters, GGUF format) also highlighted the practical trade-offs of running large open-source models locally: multiple quantization levels are offered specifically to fit different hardware/VRAM budgets, from ~10GB (2-bit) up to ~26GB (6-bit), which directly affects whether a model can run on Colab's free-tier GPU at all.